

Knowledge Representation and Reasoning

National University of Science and Technology POLITEHNICA Bucharest
Department of Computer Science

Fall 2026

Adina Magda Florea

Lecture 2

Description Logics (DL)

RDF and RDFS

Web Ontological Language (OWL)

PART I. DESCRIPTION LOGICS

Description Logics

Description Logics (DLs) - knowledge representation formalisms that model domain knowledge

- **concepts** of the domain → terminology
- **individuals** → the world description

- Achieves **classification** of
  - concepts - subconcept/superconcept relationships (subsumption)
  - individuals - whether a given individual is an instance of a concept

- Use formal model-theoretic semantics

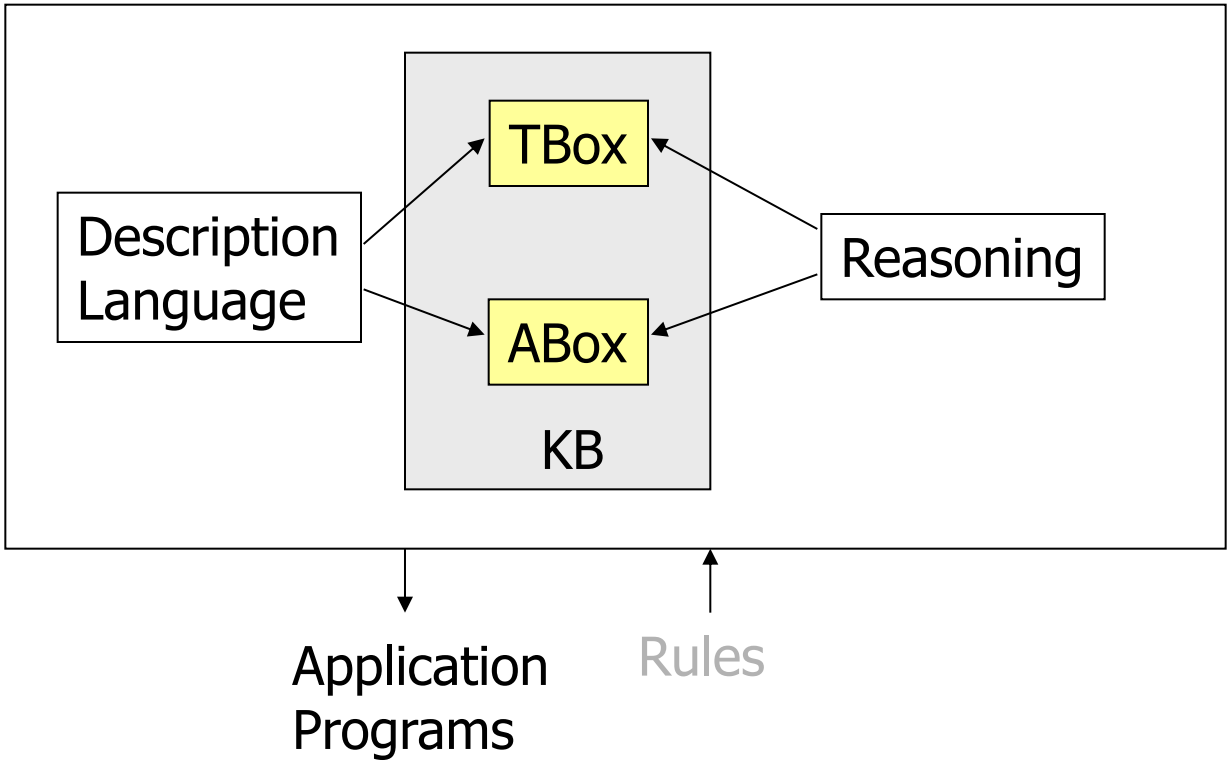
Why Description Logics?

- Connect DL expressions to OWL 2 syntax and semantics, and automated reasoning
- Application areas:
  - **Ontology engineering** (e.g., medical ontologies)
  - **Semantic Web technologies** (OWL, RDF)
  - **Knowledge graphs** (e.g., Google Knowledge Graph)
  - **AI reasoning systems**

A DL system

TBox (Terminological Box) – **concepts** (set of individuals) + **roles**: vocabulary, e.g., Mother $\sqsubseteq$ Person

ABox (Assertional Box) – facts about **individuals** in terms of the vocabulary , e.g., Person(alice), hasChild(alice,bob)



DL Syntax

C, D →	A |	(atomic concept)	
	T |	(universal concept)	Top concept
	⊥ |	(bottom concept)	Bottom concept
	¬A |	(atomic negation)	
	C⊓D |	(intersection)	Concept constructors
	∀R.C |	(value restriction)	
	∃R.T |	(limited existential quantification)	
	C⊔D |	(union)	
	∃R.C |	(full existential quantification)	
	¬C |	(negation of concepts)	
	≤ n R |	(number restrictions)	
	≥ n R		

Interpretation and models

- An **interpretation** $I = (\Delta^I, \cdot^I)$ has nonempty domain Δ^I and an interpretation function
- Concept $A^I \subseteq \Delta^I$; role $r^I \subseteq \Delta^I \times \Delta^I$; individual $a^I \in \Delta^I$

- I is a model of ontology O if I satisfies every axiom of O

- *An individual name need not identify a distinct entity (no unique-name assumption)*
- Two different names may refer to the same object. The reasoner never assumes that :Ann and :Mary are different people just because they have different names. Unless stated otherwise, it leaves open the possibility that they're the same individual.
- dbpedia:Bucharest, wikidata:Q19660 and geonames:683506 all denote one city

Semantics of DL constructors

$$\top^{\mathcal{I}} = \Delta^{\mathcal{I}} \qquad \bot^{\mathcal{I}} = \emptyset$$

$$(\neg C)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$$

$$(C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}} \qquad (C \sqcup D)^{\mathcal{I}} = C^{\mathcal{I}} \cup D^{\mathcal{I}}$$

$$(\exists r.C)^{\mathcal{I}} = \{x \mid \exists y: (x,y) \in r^{\mathcal{I}} \text{ and } y \in C^{\mathcal{I}}\}$$

$$(\forall r.C)^{\mathcal{I}} = \{x \mid \forall y: (x,y) \in r^{\mathcal{I}} \text{ implies } y \in C^{\mathcal{I}}\}$$

Restrictions & Axioms

Existential vs universal restrictions

- ∃hasChild.Doctor - at least one child is a Doctor
- ∀hasChild.Doctor - all children (if any) are Doctors
- The second expression is true even for someone with no children (vacuous truth)
- To require one or more Doctor children and only Doctor children: ∃hasChild.Doctor ⊓ ∀hasChild.Doctor

Necessary vs sufficient axioms

- Mother ⊑ Person ⊓ ∃hasChild.Person - every Mother meets these necessary conditions
- Mother ≡ Person ⊓ Female ⊓ ∃hasChild.Person - necessary AND sufficient
- A definition with ≡ enables classification in both directions

Inconsistency / unsatisfiable class

- Woman ⊑ Person ; Man ⊑ Person ; DisjointClasses(Woman, Man)
- ABox: Woman(alex) and Man(alex) → **ontology inconsistency**
- Class Impossible ≡ Woman ⊓ Man is **unsatisfiable**, even when no individual is asserted
- *The distinction: ontology inconsistency vs class unsatisfiability*

Core DL reasoning tasks

- **Ontology consistency** - does at least one interpretation satisfy ALL axioms?
- **Concept satisfiability w.r.t. O** - can C have at least one instance in a model?
- **Subsumption** $O \models C \sqsubseteq D$ means every C is necessarily a D
- **Instance checking** $O \models C(a)$ means a belongs to C in all models

- *Classification and satisfiability compute class and instance hierarchies*

Reducing reasoning problems

$\mathcal{O} \models C \sqsubseteq D$ iff $C \sqcap \neg D$ is unsatisfiable w.r.t. $\mathcal{O}$

C is unsatisfiable iff $\mathcal{O} \models C \sqsubseteq \bot$

$\mathcal{O} \models C(a)$ iff $\mathcal{O} \cup \{\neg C(a)\}$ is inconsistent

Entailment means truth in EVERY model—not just in one

Cardinality

- ≥ 3 hasChild.Person requires three DISTINCT fillers
- hasChild(mary,bob), hasChild(mary,helen), hasChild(mary,tom) alone do not prove ≥ 3 because OWL lacks a unique-name assumption
- Assert DifferentIndividuals(bob, helen, tom) or other axioms to establish the difference

PART II. RDF AND RDFS

The evolution of Web

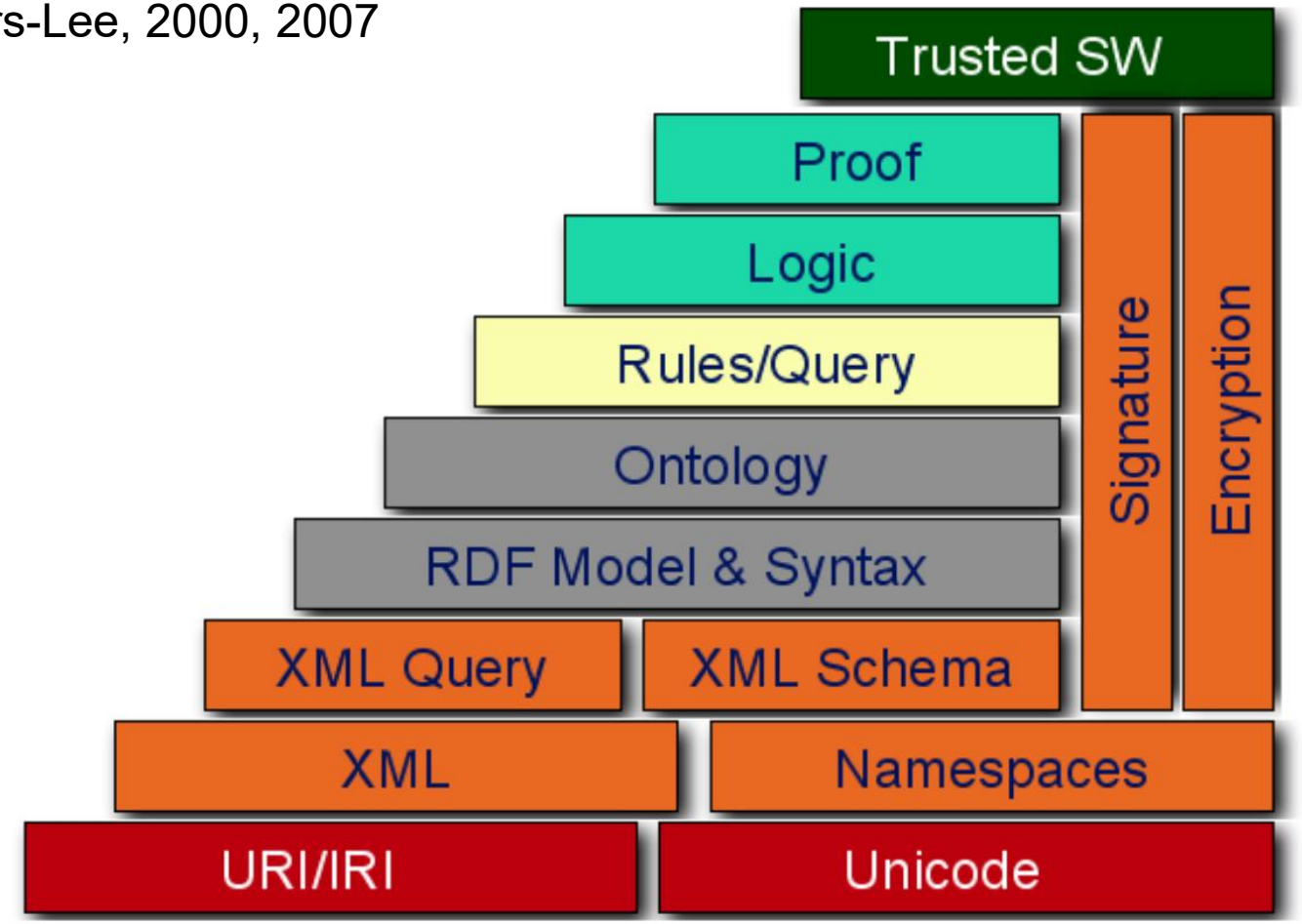
- Web 1.0: documents (static content)
- Web 2.0: social & dynamic web (user interaction)
- **Web of Data / Semantic Web**: data + semantics (machines can integrate meaning)
  - an extension of the standard World Wide Web overseen by the W3C (World Wide Web Consortium)
- Not to be confused with "**Web3 (Blockchain Web)**", which today usually means the blockchain-based web
  - a term popularized around 2021 for a decentralized internet built on blockchain technology, cryptocurrencies, smart contracts, and tokenomics

What is the Semantic Web?

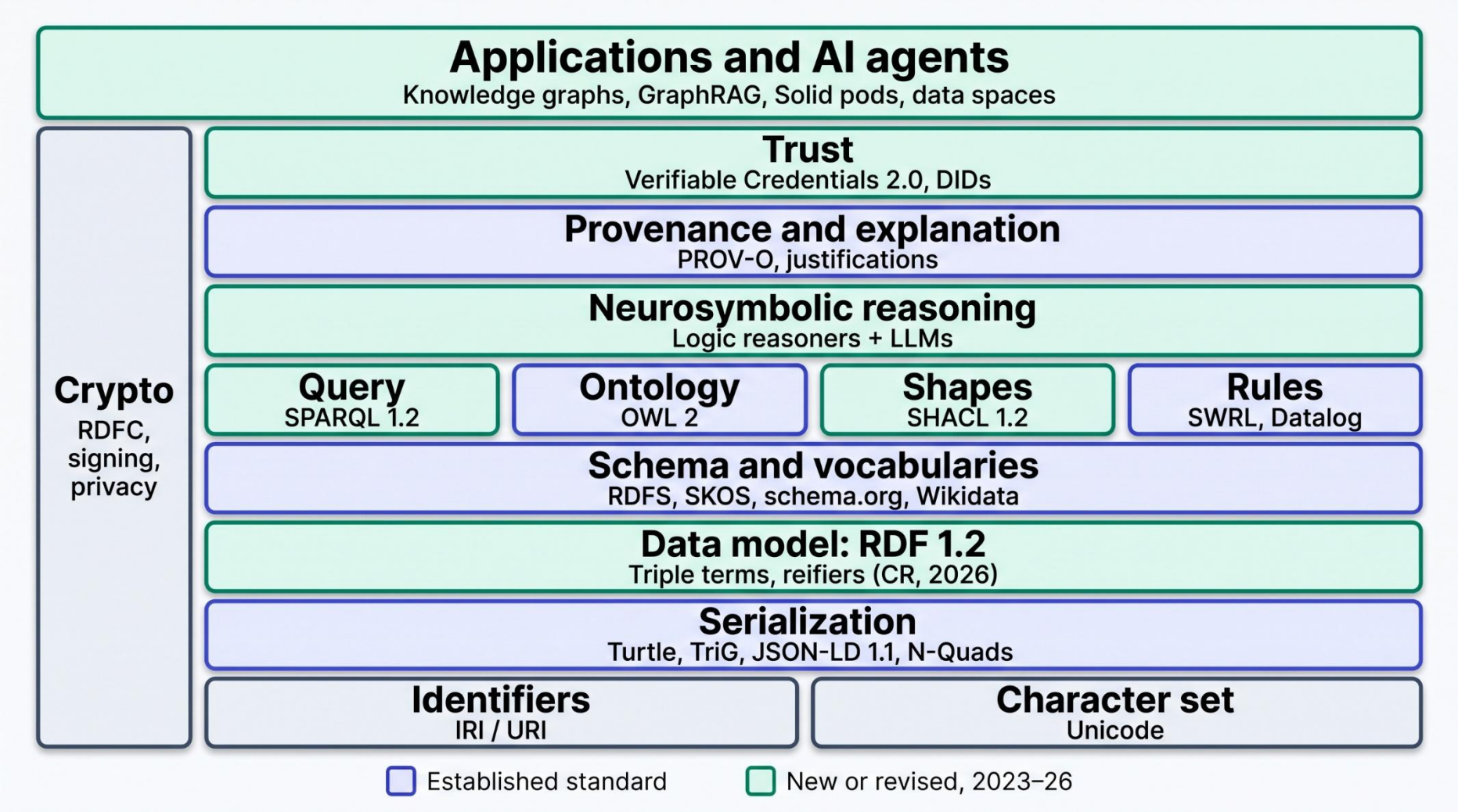
- Goal: from pages to knowledge (knowledge graphs)
- A web of data linked by meaning (semantics)
- Built on open W3C standards (the stack: next slides)
- Enables interoperability and automated reasoning
- Deployed today mainly as knowledge graphs: Wikidata, schema.org enterprise and biomedical KGs

Semantic Web Stack

Semantic Web “layer cake”
Tim Berners-Lee, 2000, 2007



Semantic Web Stack – how it can be depicted in 2026



URI - Uniform Resource Identifier, IRI - Internationalized Resource Identifier

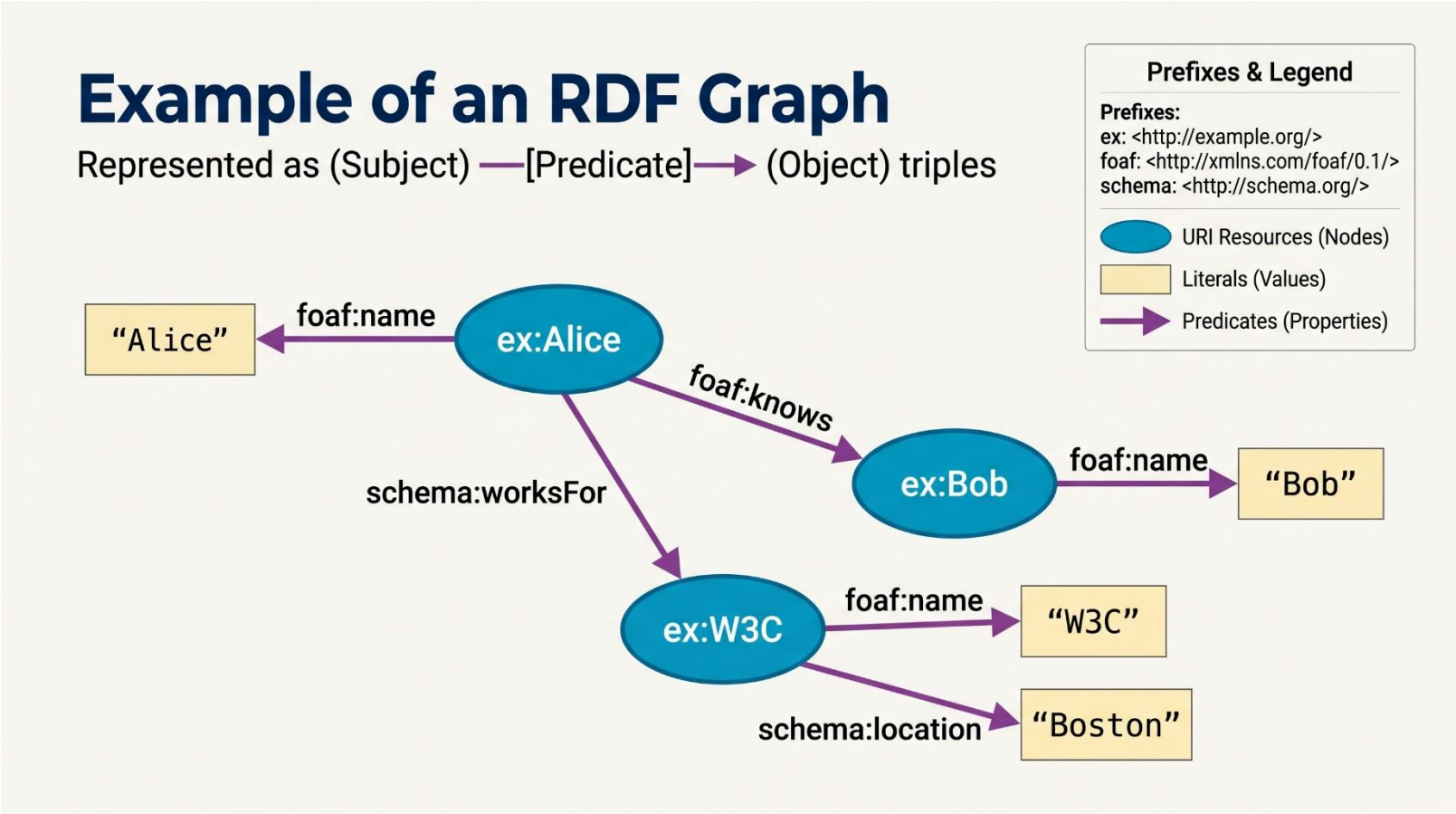
RDF

- An RDF document is a set of statements or triples
- Statements are <subject, predicate, object> triples
  - ***Subject****:* a resource
  - ***Object****:* a resource or a literal
  - ***Predicate:*** a resource
- Subjects/Predicates are typically URIs; objects can be URIs or literals

- URI is used for identifying (naming) resources on the Web by providing a **unique name for each resource**
- URIs are unambiguous
- Web provides global namespaces for URIs

RDF Graph Model

- Nodes - resources or literals; Edges - predicates (properties)
- Triples connect nodes into a directed labeled graph



Turtle syntax

- Turtle is the most readable RDF syntax
- Contains prefix declarations, then triples ending in a full stop
- The semicolon repeats the subject: Alice has a *fullName* and a *hireDate*. A comma would repeat subject and predicate. The typed literal "2024-01-10"^^xsd:date shows how datatypes are written
- Every prefix used must be declared; the xsd: prefix line is needed for the date.
- *worksFor* and *hasEmployee* are inverses: here both directions are stated explicitly. In OWL we could declare them inverse properties and let the reasoner infer one from the other.

```
@prefix ex:  <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Alice ex:worksFor ex:CompanyX .
ex:CompanyX ex:hasEmployee ex:Alice .
ex:Alice ex:fullName "Alice Smith" ;
        ex:hireDate "2024-01-10"^^xsd:date .
```



JSON-LD – Example

- **JSON-LD** - JavaScript Object Notation for Linked Data
- A lightweight, W3C-standard method of encoding Linked Data using standard JSON (JavaScript Object Notation)
- JSON-LD is RDF written as JSON. The @context maps short JSON keys to IRIs; @id gives the subject's IRI.
- Web developers can use it without knowing RDF: it is ordinary JSON. That is why *schema.org* markup in web pages uses JSON-LD, and search engines read it
- JSON-LD 1.1 is the current W3C Recommendation (2020).

```
{
  "@context": {"ex": "http://example.org/"},
  "@id": "ex:Alice",
  "ex:worksFor": {"@id": "ex:CompanyX"},
  "ex:fullName": "Alice Smith"
}
```

RDF 1.2: Statements about Statements

- Problem: how do we say who asserted a triple, when, and with what confidence?
- While RDF 1.1 treated the graph as basic, flat triples (Subject → Predicate → Object) RDF 1.2 introduces **triple terms: a triple can itself be the object of another triple**
- This allows to treat an entire triple as a node and make statements about that statement (reification/provenance)
- Turtle 1.2 annotation syntax attaches metadata to an asserted triple

- Status: W3C Candidate Recommendation (April 2026)

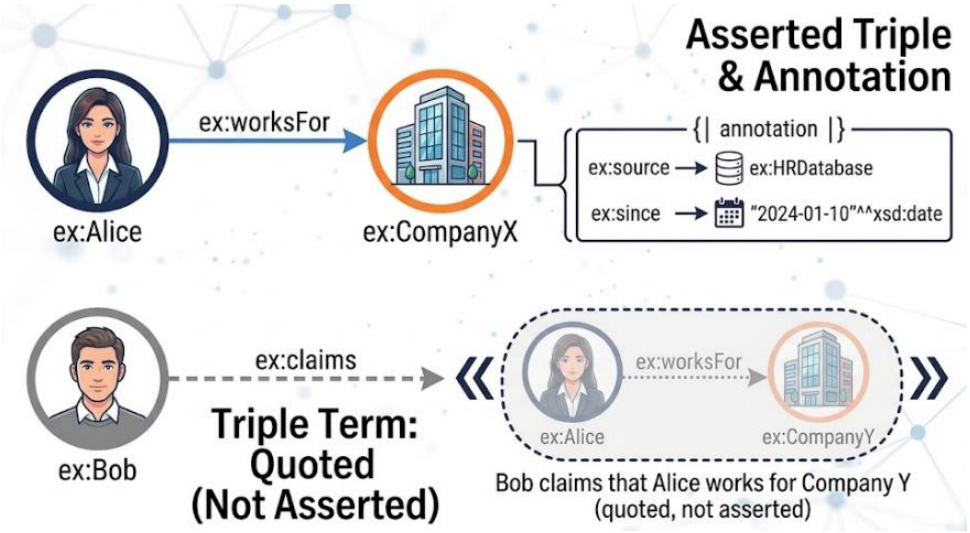
RDF 1.2 – Turtle Example

```
@prefix ex:  <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# asserted triple + annotation
ex:Alice ex:worksFor ex:CompanyX
    {| ex:source ex:HRDatabase ;
       ex:since "2024-01-10"^^xsd:date |} .
```

```
# triple term: quoted, not asserted
ex:Bob ex:claims <<( ex:Alice ex:worksFor ex:CompanyY )>> .
```

Property graphs offer edge metadata natively; RDF 1.2 closes this gap



RDFS

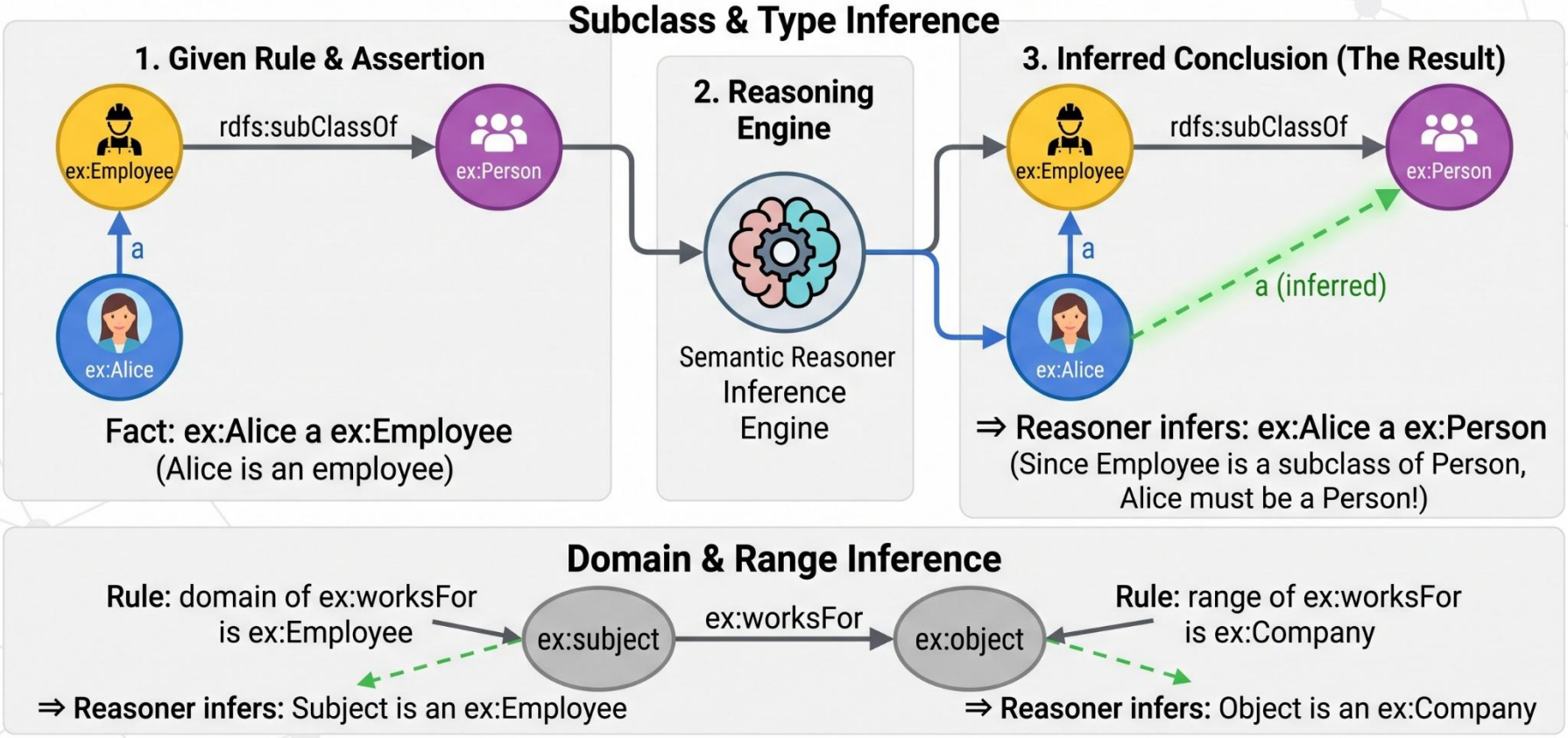
- RDFS: Schema Vocabulary
- rdfs:Class, rdfs:subClassOf
- rdf:Property, rdfs:domain, rdfs:range
- rdfs:label, rdfs:comment (annotations)

RDFS Inference

- Given: ex:Employee rdfs:subClassOf ex:Person
- If ex:Alice a ex:Employee ⇒ reasoner infers ex:Alice a ex:Person
- Domain/Range: using rdfs:domain/range to infer types

RDFS Inference

Deducing implicit facts from explicit data



AI generated image

PART III. OWL2



OWL 2

Web Ontology Language

- Current standardized family is OWL 2
- OWL 2 DL: decidable, supported by reasoners
- OWL 2 Full: RDF-based semantics, undecidable
- (OWL 1 Lite / DL / Full are historical)

OWL 2 Profiles

OWL EL - large taxonomies

- SNOMED CT is the largest clinical terminology, with several hundred thousand concepts. It uses the EL profile: conjunctions and existential restrictions, but no universal restrictions, negation or disjunction
- EL classification is polynomial → reasoners such as ELK classify the whole of SNOMED CT quickly. This is industrial strength: it powers coding and decision support in healthcare systems

OWL QL - query answering over large relational data

- Designed for data that stays in relational databases. Queries over the ontology are rewritten into SQL. This is the basis of *virtual knowledge graphs*, or ontology-based data access. Ontop is the reference system (an open-source system for virtual knowledge graphs, also called ontology-based data access, OBDA)

OWL RL - rule-oriented inference

- Designed for rule engines: its semantics can be implemented by forward-chaining rules

The profile is a design tradeoff - expressivity vs tractability, not an accuracy ranking

OWL 2

OWL basic components

- Classes & Individuals
- Properties
  - Object properties (between instances/individuals of 2 classes)
  - Data properties (between class instances/individuals and data types)
- Annotation properties (labels, comments)
- owl:Thing (T), owl:Nothing (⊥)

- DL Concepts = OWL Classes
- DL individuals = OWL Individuals
- DL Roles = OWL Properties

Manchester ↔ DL correspondences

and ↔ ⊓ or ↔ ⊔ not ↔ ¬

some ↔ ∃ only ↔ ∀

SubClassOf ↔ ⊑ EquivalentTo ↔ ≡

min n ↔ ≥ n max n ↔ ≤ n exactly n ↔ = n

hasChild some Person ↔ ∃hasChild.Person

hasChild only Person ↔ ∀hasChild.Person

hasChild min 2 Person ↔ ≥ 2 hasChild.Person

hasChild exactly 1 ↔ = 1 hasChild.⊤ (unqualified)

OWL 2 Example in Manchester Syntax

```
Prefix: : <http://example.org/university#>
Ontology: <http://example.org/university>

Class: Person

Class: Student
   SubClassOf: Person

Class: Professor
   SubClassOf: Person

Class: Course

ObjectProperty: attends
   Domain: Student
   Range: Course

ObjectProperty: teaches
   Domain: Professor
   Range: Course

Class: EnrolledStudent
   EquivalentTo:
      Student and (attends some Course)
```

```
Individual: Alice
   Types: Student
   Facts: attends AI_Course

Individual: Bob
   Types: Professor
   Facts: teaches AI_Course

Individual: AI_Course
   Types: Course
```

OWL2 reasoner can infer Alice is EnrolledStudent

If a student has no explicitly stated *attends* relationship, we cannot conclude that the student is not an EnrolledStudent. The ontology may simply contain incomplete information.

OWL2 Example in Turtle

```
@prefix : <http://example.org/university#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
# Ontology
<http://example.org/university> a owl:Ontology .

# Classes
:Person a owl:Class .
:Student a owl:Class ;
    rdfs:subClassOf :Person .
 :Professor a owl:Class ;
    rdfs:subClassOf :Person .
 :Course a owl:Class .

# Object Properties
:attends a owl:ObjectProperty ;
    rdfs:domain :Student ;
    rdfs:range :Course .
:teaches a owl:ObjectProperty ;
    rdfs:domain :Professor ;
    rdfs:range :Course .

# Equivalent Class
:EnrolledStudent a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            :Student
            [   a owl:Restriction ;
                owl:onProperty :attends ;
                owl:someValuesFrom :Course    ]    )   ]
.

# Individuals
:Alice a owl:NamedIndividual, :Student ;
    :attends :AI_Course .
 :Bob a owl:NamedIndividual, :Professor ;
    :teaches :AI_Course .
 :AI_Course a owl:NamedIndividual, :Course .
```

Properties relationships

- EquivalentClasses(A,B) asserts mutual subsumption
- DisjointClasses(A,B) means empty intersection
- ObjectProperty vs DataProperty: entity-to-entity vs entity-to-literal

ObjectProperty: madeFromGrape
Domain: Wine
Range: WineGrape

DatatypeProperty: yearValue
Domain: VintageYear
Range: xsd:positiveInteger

$\exists$ madeFromGrape.T $\sqsubseteq$ Wine

$\exists x\ madeFromGrape(y,x) \rightarrow Wine(y)$

T $\sqsubseteq$ $\forall$madeFromGrape.WineGrape

$\forall x\ madeFromGrape(y,x) \rightarrow WineGrape(x)$

Property axioms

- InverseObjectProperties(hasChild,hasParent) xR_1y iff yR_2x
- SymmetricObjectProperty(relatedTo) xRy iff yRx
- TransitiveObjectProperty(ancestorOf) xRy and yRz imply xRz
- FunctionalObjectProperty(hasBirthDateRecord) xRy and xRz imply y = z

Reasoning tasks

- Ontology classification (compute subclass lattice)
- Satisfiability checking (is a class consistent?)
- Realization/stisfiability (classify individuals)
- Entailment/logical consequence and query answering

Reasoning tasks - example

- Assert
  - Mary a Mother
  - Mary hasChild John
  - Woman SubClassOf Person
  - hasChild Range Person
  - Mother EquivalentTo Woman and (hasChild some Person)
  - Parent EquivalenTo Person and (hasChild some Person)
- Reasoner infers
  - Mary a Person
  - Mary a Parent
  - John a Person
  - Mother SubClassOf Parent
- Demonstrates implicit knowledge discovery

Necessary and sufficient conditions

- **Necessary conditions** define the conditions that an individual has to fulfill in order to be an instance of a concept

  Mother SubClassOf Fem and hasChild some Pers

  Partially defined Class (concept)

- **Necessary and suficient conditions** define the conditions that, if an individual fulfills, then the individual is an instance of a concept

  Mother ≡ Fem and hasChild some Pers

  Totally defined Class (concept)

Open World Assumption

- Missing facts are unknown, not automatically false
- Under the **open-world assumption**, the absence of a statement does not imply its negation
- OWL supports explicit negative property assertions; it does not use database-style negation as failure
- Example

  Horse and (hasRider some Woman)

  Horse and (hasRider some Man)

  May have also Child as hasRider

$$\text{Horse} \sqcap \exists \text{hasRider.Woman}$$
$$\text{Horse} \sqcap \exists \text{hasRider.Man}$$

- Closure axiom (to "close" the world)

  Horse and hasRider only (Woman or Man)

$$\text{Horse} \sqcap \forall \text{hasRider.}(\text{Woman} \sqcup \text{Man})$$

Why OWL uses OWA?

- OWL is for the Web: knowledge is distributed and incomplete. OWA lets ontologies be safely combined without causing accidental contradictions from omissions

OWL 2 /RDF data validation: SHACL

SHACL - Shapes Constraint Language

- OWL works under open-world assumption
- OWL infers new facts; a missing value is not an error
- SHACL: closed-world checks of the data that are actually in the KB
- SHACL asks whether an RDF graph conforms to constraints (evaluated against available data).
- Shapes select target nodes (sh:targetClass) and constrain their properties
- Constraints: sh:minCount, sh:maxCount, sh:datatype, sh:class, sh:pattern
- Output: a validation report listing each violation
- SHACL 1.0: W3C Recommendation (2017); SHACL 1.2 in W3C drafts (2026)

SHACL

- In OWL axiom Person ⊑ (hasAge some xsd:integer) - an age value must exist in all models, possibly unnamed
- It does not establish that an explicit age literal occurs in the stored graph

- In SHACL minCount 1 checks that the selected graph includes a matching value
- Use OWL for logical entailment; SHACL for operational data-quality checks

SHACL example in Turtle

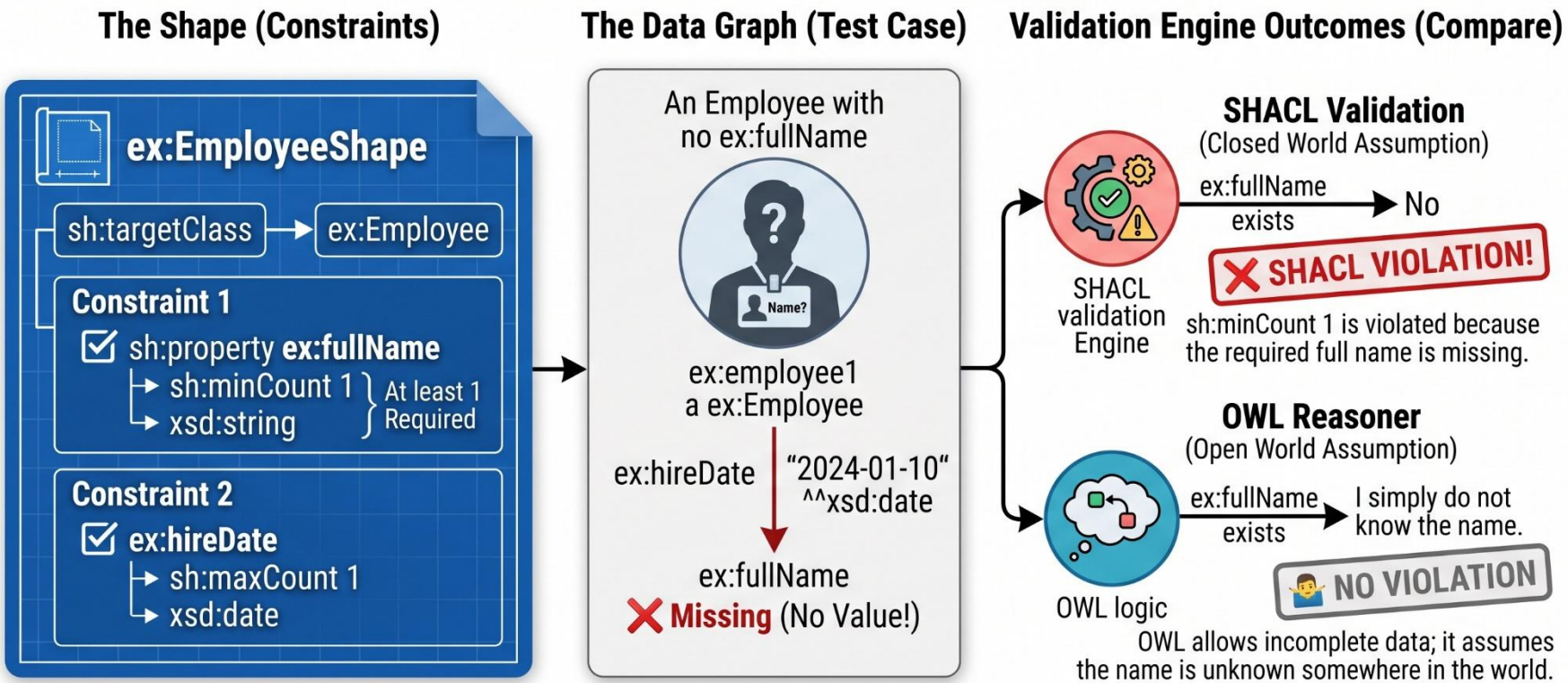
```
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix ex:  <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:EmployeeShape a sh:NodeShape ;
  sh:targetClass ex:Employee ;
  sh:property [ sh:path ex:fullName ;
        sh:minCount 1 ; sh:datatype xsd:string ] ;
  sh:property [ sh:path ex:hireDate ;
        sh:maxCount 1 ; sh:datatype xsd:date ] .
```

An Employee with no ex:fullName → violation (OWL would simply not know the name)

SHACL Constraint Validation

Checking RDF data against Shape rules (ex:EmployeeShape) and comparing SHACL with OWL when data is missing.



An Employee with no *ex:fullName* → **violation** (OWL would simply not know the name)

AI generated image

Ontology tools

- Protégé: ontology editing and reasoner integration
- HermiT, FaCT++, Openllet are OWL DL reasoning with varying feature support and maintenance
- ELK is efficient OWL 2 EL reasoning - classification (computing the full class hierarchy) runs in polynomial time, whereas full OWL 2 DL is far harder
- RDF stores support selected RDFS/OWL rule entailment - triple stores such as GraphDB, Apache Jena, do not run a full OWL reasoner. They apply a chosen subset of inference rules

Practical use cases

- **Biomedical:** GO (Gene Ontology), ChEBI (Chemical Entities of Biological Interest), HPO (Human Phenotype Ontology), Mondo (Mondo Disease Ontology), DO (Disease Ontology), CARO (Common Anatomy Reference Ontology)
- **SNOMED CT** - the largest clinical terminology, with about 350k+ concepts. Contains: findings, disorders, procedures, body structures, substances and more, and is used for coding data in electronic health records. It is licensed rather than fully open
- **Open knowledge graphs**: Wikidata, with over 100 million items, acts as a hub for linked data
- **Enterprise knowledge graphs** use OWL for alignment and SHACL for validation
- **Digital twins** also process ontologies
- **Research** - ontology embeddings such as OWL2Vec* bring ontologies into machine learning

Knowledge Graphs + LLMs

- **The complementarity.** LLMs are fluent but can hallucinate; knowledge graphs are precise but hard to query for non-experts
- ***Retrieval approaches*** such as GraphRAG ground answers in facts from the graph
  - GraphRAG = any RAG over a knowledge graph. Instead of only retrieving text passages by embedding similarity, the system retrieves entities, relations, paths or subgraphs and gives them to the LLM as context
- ***Text-to-query systems*** translate natural-language questions into graph queries. The provenance stored in the graph supports explanations, which connects to explainable AI
- ***Neuro-symbolic systems*** go further - reasoners check LLM output for consistency with the ontology

LLMs for Ontology Engineering

- LLMs are now used to help build ontologies
- **LLMs as ontology engineering assistants**
  - *Term & concept extraction*: extract candidate classes and terms from domain text
  - *Taxonomy building*: propose *is-a* hierarchies and place new terms in existing ontologies
  - *Relation extraction*: suggest non-taxonomic links (part_of, treats, causes)
  - *Ontology matching*: align concepts across ontologies (e.g., Mondo ↔ SNOMED CT)
  - *Text-to-axioms:* translate natural-language statements into OWL or structured schemas
- The essential idea is that outputs must be verified
- Reasoners check consistency, SHACL checks data quality, and domain experts check meaning
- Formal semantics is what allows verification of what an LLM produces

Example

From a sentence to verified ontology axioms

Input text "Metformin is an oral antidiabetic drug used to treat type 2 diabetes."

1. LLM extracts terms, types and relations

Term	Type	Relation
Metformin	Chemical entity	has_role → Antidiabetic drug
Type 2 diabetes	Disease	—
		Metformin → treats → Type 2 diabetes

2. LLM drafts OWL

Class: Metformin

SubClassOf: ChemicalEntity

has_role some AntidiabeticDrug

treats some Type2Diabetes

3. Grounding. Terms are linked to existing ontology IDs (Metformin → ChEBI, Type 2 diabetes → Mondo)

4. Verification

- **Reasoner:** if the LLM had instead proposed Metformin SubClassOf Disease, the reasoner flags it as unsatisfiable, because ChemicalEntity and Disease are declared disjoint
- **SHACL:** checks that treats connects a drug to a disease
- **Domain expert:** approves or rejects the axiom